

# SkillRAE: Agent Skill-Based Context Compilation for Retrieval-Augmented Execution

Xiangcheng Meng Shu Wang Yixiang Fang\*

The Chinese University of Hong Kong, Shenzhen

xiangchengmeng@link.cuhk.edu.cn, shuwang3@link.cuhk.edu.cn, fangyixiang@cuhk.edu.cn

## Abstract

Large Language Model (LLM)-based agents (e.g., OpenClaw) increasingly rely on reusable skill libraries to solve artifact-rich tasks such as document-centric workflows and data-intensive analysis. As these libraries grow, a few works have attempted to study the Retrieval-Augmented Execution (RAE), which often first retrieves some external skills and other knowledge, then compiles the context using retrieved skills, and finally executes the task. Existing works mainly focus on optimizing skill retrieval and task execution, and they pay little attention to how to effectively organize the selected skill evidence in a form that is compact, grounded, and immediately usable for the downstream executors to complete tasks. To fill this gap, we propose SKILLRAE, a two-stage RAE approach focusing on skill-based context compilation, which consists of the offline and online stages. Specifically, in the offline indexing stage, it builds a multi-level skill graph over skill communities, skills, and reusable subunits, for capturing their relationships. In the online retrieval stage, it first performs skill-ranked retrieval with selected-subunit evidence export in the graph, and then applies rescue-aware compact compilation to recover the key evidence. Together, these components compile a coarse-ranked skill set into a task-specific context that is compact, grounded, and immediately usable. Experiments on two public benchmarks show that SKILLRAE achieves a significant improvement over baselines for RAE. For example, on SkillsBench, it achieves an improvement of **11.7%** over the SOTA method. Ablation studies further show that our context compilation is crucial, instead of a mere prompt addition.

## 1 Introduction

LLM-based agents, such as OpenClaw and Manus, are increasingly deployed as capability-access systems to complete artifact-rich tasks such as document-centric workflows and data-intensive analysis [1–6]. Instead of relying only on parametric model knowledge, they often use pre-defined agent skills to execute tasks. Conceptually, an agent skill is a reusable procedural artifact that encode the specific “how-to” knowledge for coordinating tools, memory, and runtime context under concrete constraints [7–9]. When there is no ambiguity, we simply call it a skill. Figure 1 shows a real citation-management skill from SkillsBench, illustrating that an agent skill is a reusable bundle rather than a single tool name. As shown in recent works, many skill libraries, such as SkillBench [8] and AgentSkillIOS [9], have been constructed and released.

Recently, a few works have attempted to study the *Retrieval-Augmented Execution (RAE)*, which often first retrieves some external skills and other knowledge, then compiles the context using retrieved skills, and finally executes the task. These works can roughly be classified into three categories: 1) Skill routing methods focus on identifying relevant APIs or skills from large candidate pools [3–6, 10]. 2) Repository-aware retrieval methods show that code, document, and graph structure can

---

\*Corresponding author.



Figure 1: An example agent skill from SkillsBench. It contains a natural-language description, together with optional linked files such as assets, references, and scripts.

improve evidence access beyond flat vector search [11–18]. 3) Execution planning methods further decompose tasks and assemble multi-step tool-use trajectories or execution plans [19, 20]. However, these works mainly focus on optimizing skill retrieval and task execution, and they pay little attention to skill-based context compilation and face a key issue: *A retrieved skill set may be globally relevant, but still under-resolved for execution.* This is because skill routing methods usually stop at ranked skills, repository-aware retrieval methods improve evidence access without compiling a skill-facing context artifact, and execution planning methods often change the execution surface by introducing explicit plans or control policies. Hence, what remains missing is a compilation mechanism that can effectively convert the selected skill evidence in a form that is compact, grounded, and immediately usable for the downstream executors to complete tasks.

This gap is nontrivial for two reasons. First, individual skills are often designed for customizing some specific tasks. While many real-world tasks are highly relevant to these skills, they cannot use them directly, since the subunits (e.g., short procedure, file convention, usage note, etc.) of a skill may be critical for solving many tasks, leading to high relevance. Second, isolated subunits of skills cannot be used for solving a task either, though they may be relevant to it, because the subunits of a skill often have dependence relationship with some constraints [12, 13, 17, 18]. Here, a *subunit* refers to a small yet reusable unit extracted from a skill, such as a script, an execution step, or a procedural instruction. For example, the “generate properly formatted BibTeX entries” in Figure 1 can be regarded as a subunit. Hence, the key challenge is not just select which individual skills or subunits are relevant, but how to effectively compile the relevant skills in a form that is compact, grounded, and immediately usable.

To fill the above gap, we propose SKILLRAE, a two-stage RAE approach for compiling the skill-based context; that is, given a task request and a repository of reusable skills, we aim to identify a set of relevant skills and subunits, and then compile them into a task-specific context that is compact, grounded, and immediately usable. Specifically, SKILLRAE consists of an offline stage and an online stage. In the stage, it builds a multi-level skill graph that includes skill nodes, subunit nodes, skill descriptions, skill communities, and their link relationships. Serving as a powerful repository, this graph can well capture the relationships between these different nodes. In the online stage, SKILLRAE first retrieves some relevant skills. Next, it identifies a small number of highly relevant subunits from other skills that are not selected. Afterwards, to make the context compact, grounded, and directly reusable for agents, it synthesizes the identified subunits into the most relevant retrieved skills. Finally, it serializes the retrieved skills into a task-specific context with some task-specified guidance on how to apply these skills effectively.

We have conducted extensive experiments on two public benchmarks, and the results show that SKILLRAE significantly outperforms the SOTA baselines. For example, on SkillsBench with 87 tasks, SKILLRAE achieves a mean verifier reward of 29.26%, improving over pure curated skills by 11.7% relative and over the strongest automated baseline by 32.8% relative. Ablation studies further show that context compilation is crucial for forming an executable skill context, rather than serving as a simple prompt addition.

In summary, our main contributions are summarized as follows:

- We propose SKILLRAE, a two-stage RAE approach that builds a multi-level skill graph in the offline stage, and retrieves skills and compiles the context in the online stage.

- We propose a novel context compilation method, which compiles both the highly relevant skills and subunits of skills and generates some task-specified guidance.
- We extensively evaluate SKILLRAE from different angles through downstream agent benchmarks, and the results show that SKILLRAE achieves superior performance.

## 2 Related Work

**Agent Skills.** A skill is a reusable procedural artifact with bounded scope that externalizes task-focused know-how, including when to act, how to execute, how to judge completion, and how to expose instructions progressively at execution time [21, 22]. This makes skills different from APIs or tool-use interfaces, which mainly specify callable operations rather than reusable execution guidance. Recent works have begun to construct skill libraries, such as SkillsBench [8] which provides an execution-centric benchmark with deterministic verifiers and AgentSkillOS [9] which organizes skills at ecosystem scale under structured orchestration pipelines. Besides benchmark construction, some works have studied how agents accumulate reusable skills and how dependency-aware repositories can support skill combination and task routing [23, 10].

**Tool and API Selection.** Tool use and API calls extend agents by giving them callable external operations for information access, computation, and environment interaction. Tool-use methods establish prompting and training paradigms for deciding when to call tools [24, 25]. API-call works further scale this process by indexing tool names, descriptions, schemas, documentation, demonstrations, and ranking candidate APIs before downstream invocation [3, 4, 26–29]. These works focus on studying how to select and invoke callable operations, whereas SKILLRAE addresses how skill evidence can be compiled into a compact, grounded, and immediately usable form.

**Retrieval-Augmented Generation (RAG).** RAG grounds LLMs by retrieving external evidence and conditioning generation on the context [30]. Existing RAG methods can be classified into three categories: vector database-based method [31], graph-based methods [17, 18, 15, 14, 32–35], and agentic methods which retrieve in an iterative action [36–39]. Note that RAG methods often have an additional post-processing step which includes passage filtering, summarization, compression, and rewriting before LLM-context placement [40–42]. For example, RECOMP [40] and LongLLM-Lingua [41] compress retrieved context, while TRACE organizes evidence into reasoning chains. Such methods retrieve factual knowledge, whereas SKILLRAE retrieves and compiles skills which contain not only knowledge, but also tool use and API calls.

**Retrieval-Augmented Execution (RAE).** RAE extends the retrieval-augmentation paradigm from knowledge-grounded generation to agent execution, where external procedural artifacts, such as skills, tools, APIs, and so on, are retrieved to execute a specific task. Existing RAE works can be grouped into three categories: 1) Skill routing methods focus on selecting, structuring, or composing reusable skills from large skill ecosystems [10, 23, 9]. 2) Repository-aware retrieval methods exploit code, dataflow, document, or graph structure to improve access to execution-relevant evidence beyond flat vector search [11–13]. 3) Execution planning methods further connect retrieval with explicit execution planning or orchestration by assembling multi-step tool-use trajectories, model calls, or skill pipelines [19, 43, 20, 9]. In contrast, SKILLRAE focuses on the compilation step between retrieval and execution, converting relevant skills and subunits into a compact, grounded, and immediately usable context, without changing the downstream executor.

## 3 Our RAE Approach: SKILLRAE

### 3.1 Overview

Given a task request  $q$ , a task environment, and a repository of reusable skills, the RAE systems often first retrieve relevant skill evidence, then compile it into a task-specific context, and finally invoke an executor to complete  $q$ . Following this general framework, we propose SKILLRAE, a two-stage approach that consists of an offline stage and an online stage, where the former one builds an index while the latter one performs skill retrieval and context compilation, as depicted in Figure 2.

Specifically, in the offline stage, we construct a multi-level skill graph  $\mathcal{G}$  over skill communities, skills, and reusable subunits. This graph preserves skills as executor-compatible anchors while exposing fine-grained subunit evidence that can later support retrieval and context compilation.

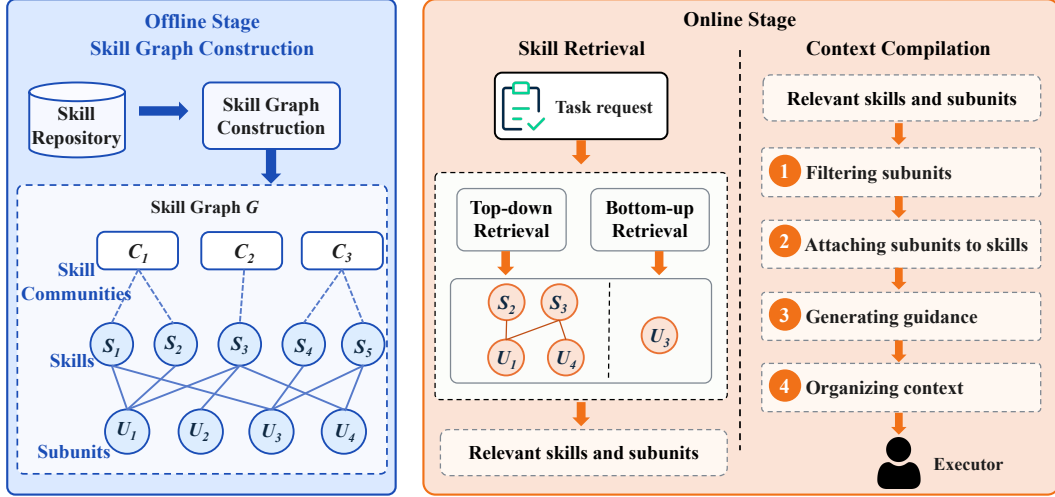


Figure 2: Overview of SKILLRAE.

In the online stage, we first perform skill retrieval over the constructed graph above by combining evidence from skill communities and subunits, which are retrieved in top-down and bottom-up manners, respectively. It then compiles the retrieved skills, selected subunit evidence, rescued subunits from non-selected source skills, and task-output constraints into a task-specific context  $\mathcal{C}(q)$ .

In the following, we introduce the details of both offline and online stages.

### 3.2 Offline Stage

In the offline stage, to capture the intricate relationships between different skills, we construct a skill graph, which is a multi-level graph representing the skill repository at three evidence levels, i.e., skill communities, skills, and subunits. Formally, we denote the repository representation as

$$\mathcal{G} = (C, S, U, E, m), \quad (1)$$

where  $C$  is the set of skill communities,  $S$  is the set of skill nodes,  $U$  is the set of normalized subunit nodes,  $E \subseteq S \times U$  denotes the edges between skills and subunits, and  $m : S \rightarrow C$  assigns each skill to one skill community. An extraction edge  $(s, u) \in E$  means that subunit  $u$  is extracted from skill  $s$ , while  $m(s)$  records the skill community assignment used later during retrieval.

During the construction, we convert the original skills into skill nodes and candidate subunits. Candidate subunits are extracted by deterministic rules over procedural lines, element references, and constraint-like usage statements; they are then normalized, filtered by length, deduplicated by exact match, and linked to their source skills through extraction edges. Additionally, we also store skill descriptions, normalized subunit nodes, subunit embeddings, and skill-subunit extraction edges.

To form skill communities, we derive a compact textual representation for each skill from high-IDF extracted subunits, embed these skill representations, and cluster the embedded skills; the resulting hard clusters serve as the skill communities, with one community assignment per skill.

In the high-level view, the skill communities can be regarded as collections of related skills, skills as executor-compatible procedures, and subunits as local procedures (e.g., file conventions and constraints). As a result, the skill communities support matching over related skills, skill nodes preserve the procedures consumed by the downstream executor, and subunits provide local procedures which are not visible from skill-level representations alone. Besides, the many-to-many extraction edges  $E$  allow normalized subunits to be shared across skills, enabling bottom-up evidence aggregation during retrieval and allowing us to incorporate task-specific information from both selected skills and rescued evidence.

### 3.3 Online Stage

In the online stage, whenever the agent receives a task request  $q$ , it first retrieves the relevant skills from the skill graph, then compiles the relevant skills into a compact, grounded, and immediately usable form, and finally invoke the executor to complete  $q$ . Next, we introduce the details of skill retrieval and context compilation sequentially.

#### 3.3.1 Skill Retrieval

Our skill retrieval method uses two signals: a top-down signal from skill communities and a bottom-up signal from subunits. The output is a selected skill set  $K(q)$ , selected-skill highlights  $H(q)$ , and non-selected rescue candidate records  $R(q)$  for context compilation.

**Top-down retrieval.** It matches the task request  $q$  to the skill communities in the multi-level skill graph, where each community is represented by a deterministic text constructed from its label and representative skills. Communities are scored by embedding similarity between  $q$  and the community text; the highest-scoring communities define the community-supported skill set  $\mathcal{B}(q)$ , which later enters skill scoring.

**Bottom-up retrieval.** It matches  $q$  to subunits and projects their evidence back to source skills. Let  $\ell_1(s, q)$  denote normalized similarity between  $q$  and the description of skill  $s$ , let  $p(s, q)$  denote a normalized skill name score, and let  $\ell_0(s, q)$  denote normalized subunit evidence for skill  $s$ . For the retained top subunits  $\mathcal{T}_N(q)$ , the projection is

$$\ell_0(s, q) = \text{norm}_S \left( \sum_{u \in \mathcal{T}_N(q)} \mathbf{1}\{(s, u) \in E\} \frac{\sigma(q, u)}{\deg(u)} \right), \quad (2)$$

where  $\sigma(q, u)$  is similarity between task  $q$  and subunit  $u$ ,  $\deg(u)$ , is the number of skills from which subunit  $u$  is extracted, and  $\text{norm}_S$  normalizes scores over skills. A subunit  $u$  contributes to a skill  $s$  only through an edge between them in the multi-level skill graph.

**Skill selection.** The final skill score combines skill-level similarity, bottom-up subunit evidence, the skill name score, and the top-down community-level weighting score:

$$\text{score}(s, q) = (\alpha \ell_1(s, q) + \beta \ell_0(s, q) + \gamma p(s, q)) (1 + \lambda \mathbf{1}\{s \in \mathcal{B}(q)\}), \quad (3)$$

where  $\mathcal{B}(q)$  is the set of skills assigned to the highest-scoring skill communities matched by the top-down retrieval direction. The selected skill set is

$$K(q) = \text{TopK}_{s \in S} \text{score}(s, q). \quad (4)$$

Skills remain the units consumed by the executor, while subunits provide evidence for scoring and later context compilation.

**Evidence export.** For each selected skill  $s \in K(q)$ , the retriever records a bounded set  $H(s, q)$  of task-relevant subunit highlights extracted from  $s$ . These highlights come from the retained subunits that contribute to  $\ell_0(s, q)$ . The retriever ranks them by contribution and task similarity, removes duplicate subunits, and keeps only a small set for each skill. They do not replace the selected skill itself; they make local procedures explicit for context compilation. Besides, the retriever keeps non-selected candidate records  $R(q)$ , which are used later for subunit filtering in the compilation stage. This design makes the retrieval stage both executor-compatible and compiler-ready: the output remains a selected skill set, while the compiler receives graph-grounded subunit evidence needed to form a task-specific context.

#### 3.3.2 Context Compilation

Given the selected skills  $K(q)$ , subunit evidence  $H(q)$ , non-selected rescue candidates  $R(q)$ , and task-output contract evidence  $\mathcal{O}(q)$ , we aim to compile them into a compact, grounded, and immediately usable form. This process does not change a source skill, planner, or executor; instead, it generates an effective context that includes task-specific guidance on how to use the selected skills

and subunit cues, together with local evidence, for the executor. As a result, the compilation process changes what the agent sees before execution, rather than the runtime orchestration.

To achieve the above goal, we develop an effective compilation method, which consists of four steps: filtering subunits, attaching subunits to skills, generating task-specific guidance, and organizing context, introduced as follows.

**Step 1: filtering subunits.** This step handles the case where a skill is not selected into  $K(q)$ , but contains a local subunit that is highly relevant to  $q$ . Because retrieval selects whole skills as the primary context unit, excluding a source skill from  $K(q)$  does not imply that all subunits extracted from that skill are useless; it only means that the skill was not selected under the aggregate skill score. To filter subunits, we scan a bounded pool of non-selected source skills and select a small intermediate set  $Z(q)$  of subunits, subject to relevance, non-redundancy, source skill alignment, and budget constraints. Each filtered subunit keeps the identifier of the non-selected source skill it was extracted from, but  $Z(q)$  is not exposed directly as a standalone context.

**Step 2: attaching subunits to skills.** The second step attaches each filtered subunit to a selected skill. Specifically, for each  $u \in Z(q)$ , we choose the selected skill with the highest compatibility:

$$s^*(u) = \arg \max_{s \in K(q)} \text{Aff}(u, s \mid q), \quad (5)$$

where  $\text{Aff}(u, s \mid q)$  is a deterministic compatibility score over a small feature vector:

$$\text{Aff}(u, s \mid q) = \omega^\top [q_{\text{rel}}(u, q), f_{\text{skill}}(u, s, H(s, q)), r_{\text{sel}}(s, q), g_{\text{same}}(u, s), g_{\text{active}}(u, s, K(q))]. \quad (6)$$

Here  $\omega$  is a fixed weight vector,  $q_{\text{rel}}$  measures lexical relevance between subunit and task,  $f_{\text{skill}}$  measures the best match between  $u$  and the selected skill  $s$  or its highlighted subunits,  $r_{\text{sel}}$  is the normalized retrieval confidence of  $s$ , and  $g_{\text{same}}$  and  $g_{\text{active}}$  encode skill community alignment. Implementation details of these features are provided in Appendix A.2.

The attachment step turns rescued subunits into selected-skill-local cues. For each rescued subunit  $u$ , the compiler records the selected skill  $s^*(u)$  to which it is attached, together with the original non-selected source skill  $p$  from which  $u$  was extracted. We denote the resulting pool of attached candidate cues as  $\mathcal{A}(q)$ . At this point,  $\mathcal{A}(q)$  is not yet the set rendered in the compiled context; it is the candidate pool passed to the guidance-generation step. This attachment step makes rescue compatible with the selected-skill interface: rescued evidence is associated with an already selected skill, not exposed as a new skill or independent execution unit.

**Step 3: Generating task-specific guidance.** This step selects which attached cues in  $\mathcal{A}(q)$  should be rendered as task-specific guidance. The compiler keeps cues that are concrete, task-aligned, non-redundant with already selected evidence, and useful under the context budget. This produces  $\mathcal{A}_{\text{final}}(q)$ , the set of affiliated cues that will be rendered in the compiled context. Low-affinity or overly generic cues are excluded even if they were recovered during rescue, preventing the compiled context from becoming a dump of all available subunit evidence.

**Step 4: organizing context.** The last step organizes the selected evidence and task-specific guidance into the compiled context.

$$\mathcal{C}(q) = \text{Compile}_B(q, K(q), H(q), \mathcal{A}_{\text{final}}(q), \mathcal{O}(q)), \quad (7)$$

where  $B$  is the context budget. The compiled context is organized around the task request, selected skills and subunits, output contract, and affiliated subunit cues. The compiled context is delivered to the downstream executor as advisory context before execution.

Overall, in the online stage, we not only retrieve the skills that are highly relevant to  $q$ , but also compile them into the context with task-specific guidance on how to apply them to complete  $q$ .

## 4 Experiments

We evaluate SKILLRAE to test whether context compilation improves downstream skill usage. The experiments compare overall performance, isolate the contribution of compilation and graph evidence, and include a targeted cross-backbone check over a second agent/model backbone.

Table 1: Benchmark and graph statistics. #L2, #L0, and #Edges denote the number of skill communities, subunits, and extracted subunits.

| Benchmark    | #Tasks | Metric      | #Skills | #L2 | #L0   | #Edges |
|--------------|--------|-------------|---------|-----|-------|--------|
| SkillsBench  | 87     | Reward Mean | 207     | 14  | 4,834 | 5,255  |
| AgentSkillOS | 30     | Score       | 200     | 14  | 718   | 918    |

#### 4.1 Setup

**Datasets.** We evaluate on two complementary skill agent benchmarks. **SKILLSBENCH** is the primary execution-centric benchmark because each task defines a deterministic verifier and produces a scalar reward, allowing us to measure downstream task completion rather than retrieval accuracy alone [8]. **AGENTSKILLOS** provides a complementary skill quality benchmark for ecosystem-scale skill use [9]. Table 1 summarizes the benchmark and graph statistics used in our experiments.

**Evaluation and metrics.** Following existing works [8, 9], we evaluate each method through end-to-end agent execution. For each method, the selected skills, along with any compiled guidance provided, are sent to the same downstream agent executor, which executes the task and produces the required output artifacts. The resulting artifacts are evaluated against the task-specific ground truth or expected constraints defined by each benchmark. For **SkillsBench**, we report the mean reward across 87 tasks, where each task is evaluated by a deterministic verifier that assigns a scalar reward based on artifact-level correctness. For **AgentSkillOS**, we report the aggregate score over 30 tasks, where weighted objective checks are mapped to a normalized score from 0 to 100. This evaluation measures the extent to which the retrieved skills and compiled guidance can be operationalized by the agent to complete the target tasks, rather than only assessing retrieval coverage.

**Baseline methods.** We compare SkillRAE with five baselines. *Benchmark-native curated skills* (from SkillsBench [8] and AgnetSkillOS [9]) denotes the curated skills that are manually specified per-task in each dataset. *Vanilla retrieval* embeds each skill using its frontmatter name and description, ranks skills by embedding similarity to the task request, and exposes the selected skills directly to the executor. *LLM-based retrieval* first uses the same embedding retriever to form a candidate pool, then asks an LLM selector to choose the final skills from candidate skill IDs, names, and descriptions; the selected skills are then exposed directly to the executor. *AgentSkillOS* [9] is evaluated in its native setting, using its built-in skill organization and orchestration pipeline; we do not add context compilation because doing so would change the execution-control surface being evaluated. *SkillRouter* [10] is evaluated as a full-text skill-routing baseline: it produces a ranked skill set, which we materialize for the same downstream executor.

**Implementation.** We build the multi-level skill graph offline from source SKILL.md files. The graph builder extracts normalized procedural, element, and constraint subunits from each skill, deduplicates them globally, and connects each retained subunit back to its source skills. We embed subunits with BAAI/bge-small-en [44] using normalized SentenceTransformer embeddings [44, 45]. Skill descriptions are embedded with the same model at retrieval time. Skill communities are constructed by embedding skill representations and applying KMeans clustering. At retrieval time, the system retrieves a bounded set of skills, exports a bounded set of highlighted subunits for each selected skill, and applies rescue-aware compilation over a bounded non-selected frontier. Exact retrieval budgets, clustering settings, rescue thresholds, and final-packet budget are provided in Appendix A.1. we use Codex CLI with GPT-5.2 [46, 47] as the downstream agent/model backbone.

#### 4.2 Overall Performance

We first compare SkillRAE with skill use and retrieval baselines under the same downstream execution setting. Table 2 reports the main comparison results. The table uses one primary metric per benchmark: SkillsBench reward mean and AgentSkillOS aggregate score.

SKILLRAE achieves the best overall performance on both benchmarks. On SkillsBench, it improves over benchmark-native curated skills by 3.06 percentage points, over SkillRouter by 7.22 percentage points, and over vanilla retrieval by 9.82 percentage points. On AgentSkillOS, it also obtains the highest aggregate score, improving over the native AgentSkillOS baseline by 1.09 percentage points and over SkillRouter by 2.29 percentage points. The strongest baselines are informative. Benchmark-native curated skills provide a strong reference because the relevant skills are manually

Table 2: Overall downstream performance. SkillsBench reports reward mean over 87 tasks. AgentSkillOS reports aggregate score under the score-based evaluation protocol.

| Method                          | SkillsBench Reward Mean (%) $\uparrow$ | AgentSkillOS Score (%) $\uparrow$ |
|---------------------------------|----------------------------------------|-----------------------------------|
| Benchmark-native Curated Skills | 26.20                                  | 78.45                             |
| AgentSkillOS                    | 17.24                                  | 83.50                             |
| Vanilla Retrieval               | 19.44                                  | 74.85                             |
| LLM-based Retrieval             | 16.34                                  | 82.06                             |
| SkillRouter                     | 22.04                                  | 82.30                             |
| SKILLRAE (Ours)                 | <b>29.26</b>                           | <b>84.59</b>                      |

specified or benchmark-defined. SkillRouter is competitive because full-text skill routing improves skill selection, while AgentSkillOS performs strongly on its native benchmark because it uses an orchestration-oriented execution pipeline. However, these methods still expose selected skills or orchestrated skill use without explicitly compiling retrieved skill evidence into a compact, task-specific execution context. The fact that SKILLRAE improves even over benchmark-native curated skills suggests that task success depends not only on selecting relevant skills, but also on how the selected skill evidence is organized for the executor.

These results indicates that effective retrieval-augmented execution requires both strong skill retrieval and context compilation. SKILLRAE improves downstream task completion by converting retrieved skills, subunit evidence, and task constraints into a compact context.

### 4.3 Detailed Analysis of SKILLRAE

**Ablation study.** To understand the effectiveness of different components in SKILLRAE, we conduct ablation studies and report the results in Table 3. The first two ablation variants remove the Bottom-up Retrieval and Top-down Retrieval modules from the Skill Retrieval stage, respectively, while the variant w/o Context Compilation removes the Context Compilation stage. Among all variants, removing Top-down Retrieval causes the largest performance degradation, reducing the score from 29.26% to 16.61% with Codex CLI + GPT-5.2 [46, 47]. This confirms the value of skill communities, which support coarse-grained matching over related skills before the agent selects and applies specific skill units. The drops observed when removing Bottom-up Retrieval further show that fine-grained matching remains useful, while the degradation under w/o Context Compilation indicates that compiling retrieved skills into a task-specified guidance is important for effective skill application.

Table 3: Component ablations across agent/model backbones on SkillsBench. Each row removes one method-level component from the full graph-native context compilation method. All entries report reward mean (%).

| Variant                   | Codex CLI + GPT-5.2 | Gemini CLI + Gemini 3 Flash |
|---------------------------|---------------------|-----------------------------|
| SKILLRAE Full             | <b>29.26</b>        | <b>28.85</b>                |
| - w/o Bottom-up Retrieval | 23.43               | 25.57                       |
| - w/o Top-down Retrieval  | 16.61               | 20.26                       |
| - w/o Context Compilation | 22.59               | 27.80                       |

**Performance under different agent backbones.** Table 3 further demonstrates the performance under different agent backbones. Table 3 also evaluates the same ablation design under two agent/model backbones: Codex CLI with GPT-5.2 [46, 47] and Gemini CLI with Gemini 3 Flash [48, 49]. The full method achieves similar rewards under the two backbone, 29.26% and 28.85%, indicating that the overall pipeline is not tied to a single agent backbone. All three ablations reduce performance under both backbones, indicating consistent component effects. At the same time, the Gemini-side drops are smaller for some components, especially Context Compilation, suggesting that the downstream agent/model backbone can partially mediate the effect size of missing context-formation signals. Nevertheless, Top-down Retrieval remains the most important module under both backbone, with the largest absolute degradation in both columns.



#### 4.4 Context Compilation on Compatible External Backbones

We next evaluate lite context compilation on compatible external retrieval backbones. A compatible backbone outputs a flat selected skill set that is consumed by the same executor. This criterion includes vanilla retrieval and LLM-based retrieval, but excludes SkillRouter and AgentSkillOS: SkillRouter is kept as a full-text routing baseline, and AgentSkillOS is kept as an orchestration baseline. Table 4 therefore tests the lite compilation interface only where the intervention is well-defined.

The context compilation results show that selected skills become more useful when they are surfaced through a compact task/output-aware exposure layer. This effect appears for both vanilla retrieval and LLM-based retrieval, where the compiler has access only to selected skills and task/output contracts. The SKILLRAE pair tests the stronger version of the same idea: given our graph-native retrieval backbone, affiliate-aware compilation further improves reward from 22.59% to 29.26%. This separates the general value of context exposure from the additional value of graph-derived subunit export and affiliate-aware rescue.

Table 4: Effect of context compilation across retrieval backbones. External backbones use lite context compilation without graph-derived affiliation. SKILLRAE Full uses graph-native affiliate-aware context compilation.

| Retrieval Backbone  | Setting               | SkillsBench Reward Mean (%) $\uparrow$ | Gain (%) $\uparrow$ |
|---------------------|-----------------------|----------------------------------------|---------------------|
| Vanilla retrieval   | No compilation        | 19.44                                  | –                   |
| Vanilla retrieval   | + Context compilation | 21.71                                  | +2.27               |
| LLM-based Retrieval | No compilation        | 16.34                                  | –                   |
| LLM-based Retrieval | + Context compilation | 23.36                                  | +7.02               |
| SKILLRAE            | Retrieval only        | 22.59                                  | –                   |
| SKILLRAE            | + Context compilation | 29.26                                  | +6.67               |

#### 4.5 Summary of findings.

The experiments support three aspects of the method. First, SKILLRAE improves over existing skill use, retrieval, routing, and orchestration baselines in the main benchmark comparison. Second, the ablations show that the gain depends on both context compilation and graph-derived evidence sources, including subunit evidence and L2 skill communities. Third, the context compilation improves compatible selected-skill exposure methods, while SKILLRAE further strengthens this exposure layer with graph-native affiliate-aware evidence. Together, these findings support the paper’s thesis that retrieval-augmented execution requires context compilation beyond skill selection: retrieved skills and subunits must be organized into a compact, grounded, and task-specific context for execution.

### 5 Limitations

SKILLRAE is most applicable to skill repositories with explicit procedural text, file/output conventions, and constraints; it may be less effective when key dependencies are hidden in opaque tools, undocumented code, or runtime state. As an advisory compiler rather than a planner or controller, SKILLRAE does not guarantee downstream context use or recovery from execution-time failures; future work can evaluate it on larger repositories, more executor backbones, and broader skill-driven benchmarks as they become available.

### 6 Conclusion

In this paper, we study the skill-based context compilation for retrieval-augmented execution (RAE). We propose SKILLRAE, a two-stage RAE approach that constructs a multi-level skill graph in the offline stage, and then in the online stage retrieves relevant skills and subunits and further compiles them into a compact, grounded, and task-specific context with task-specified guidance. Experiments on SkillsBench and AgentSkillOS demonstrate that SKILLRAE improves downstream execution performance over baseline methods. In future work, we will test SKILLRAE on more skill benchmarks and also consider more real-world constraints such as limited token budget.

## References

- [1] OpenClaw. OpenClaw: Personal ai assistant. <https://github.com/openclaw/openclaw>, 2026. Accessed: 2026-05-05.
- [2] Manus AI. Welcome – manus documentation. <https://manus.im/docs/introduction/welcome>, 2026. Accessed: 2026-05-05.
- [3] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.187. URL <https://aclanthology.org/2023.emnlp-main.187/>.
- [4] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*, 2024. URL [https://proceedings.iclr.cc/paper\\_files/paper/2024/hash/28e50ee5b72e90b50e7196fde8ea260e-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2024/hash/28e50ee5b72e90b50e7196fde8ea260e-Abstract-Conference.html).
- [5] Junjie Ye, Zhengyin Du, Xuesong Yao, Weijian Lin, Yufei Xu, Zehui Chen, Zaiyuan Wang, Sining Zhu, Zhiheng Xi, Siyu Yuan, Tao Gui, Qi Zhang, Xuanjing Huang, and Jiecao Chen. ToolHop: A query-driven benchmark for evaluating large language models in multi-hop tool use. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2995–3021, 2025.
- [6] Haiyang Shen, Yue Li, Desong Meng, Dongqi Cai, Sheng Qi, Li Zhang, Mengwei Xu, and Yun Ma. ShortcutsBench: A large-scale real-world benchmark for api-based agents. In *Proceedings of the International Conference on Learning Representations*, 2025.
- [7] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. doi: 10.48550/arXiv.2305.16291. URL <https://arxiv.org/abs/2305.16291>.
- [8] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Qunhong Zeng, Di Wang, Xuan-dong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han-chung Lee. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026. doi: 10.48550/arXiv.2602.12670. URL <https://arxiv.org/abs/2602.12670>.
- [9] Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale. *arXiv preprint arXiv:2603.02176*, 2026. doi: 10.48550/arXiv.2603.02176. URL <https://arxiv.org/abs/2603.02176>.
- [10] YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. SkillRouter: Skill routing for LLM agents at scale. *arXiv preprint arXiv:2603.22455*, 2026. doi: 10.48550/arXiv.2603.22455. URL <https://arxiv.org/abs/2603.22455>.
- [11] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2471–2484, 2023.
- [12] Wei Cheng, Yuhan Wu, and Wei Hu. Dataflow-guided retrieval augmentation for repository-level code completion. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7957–7977, 2024.

- [13] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. RepoGraph: Enhancing ai software engineering with repository-level code graph. In *Proceedings of the International Conference on Learning Representations*, 2025.
- [14] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=GN921JHCRw>.
- [15] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [16] Zhuoqun Li, Xuanang Chen, Haiyang Yu, Hongyu Lin, Yaojie Lu, Qiaoyu Tang, Fei Huang, Xianpei Han, Le Sun, and Yongbin Li. StructRAG: Boosting knowledge intensive reasoning of llms via inference-time hybrid information structurization. In *Proceedings of the International Conference on Learning Representations*, 2025.
- [17] Shu Wang, Yixiang Fang, Yingli Zhou, Xilin Liu, and Yuchi Ma. ArchRAG: Attributed community-based hierarchical retrieval-augmented generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 15868–15876, 2026. doi: 10.1609/aaai.v40i19.38619. URL <https://ojs.aaai.org/index.php/AAAI/article/view/38619>.
- [18] Shu Wang, Yingli Zhou, and Yixiang Fang. BookRAG: A hierarchical structure-aware index-based approach for retrieval-augmented generation on complex documents. *arXiv preprint arXiv:2512.03413*, 2025. doi: 10.48550/arXiv.2512.03413. URL <https://arxiv.org/abs/2512.03413>.
- [19] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in hugging face. In *Advances in Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=yHdTscY6Ci>.
- [20] Qinzhuo Wu, Wei Liu, Jian Luan, and Bin Wang. ToolPlanner: A tool augmented LLM for multi granularity instructions with path planning and feedback. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 18315–18339, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.1018. URL <https://aclanthology.org/2024.emnlp-main.1018/>.
- [21] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Xiaoxi Jiang, and Guanjuan Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026.
- [22] Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*, 2025.
- [23] Dawei Liu, Zongxia Li, Hongyang Du, Xiyang Wu, Shihang Gui, Yongbei Kuang, and Lichao Sun. Graph of skills: Dependency-aware structural retrieval for massive agent skills. *arXiv preprint arXiv:2604.05333*, 2026. doi: 10.48550/arXiv.2604.05333. URL <https://arxiv.org/abs/2604.05333>.
- [24] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL [https://openreview.net/forum?id=WE\\_vluYUL-X](https://openreview.net/forum?id=WE_vluYUL-X).
- [25] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=Yacmpz84TH>.

- [26] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems*, 2024. URL [https://proceedings.neurips.cc/paper\\_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html).
- [27] Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. ToolkenGPT: Augmenting frozen language models with massive tools via tool embeddings. In *Advances in Neural Information Processing Systems*, 2023. URL [https://proceedings.neurips.cc/paper\\_files/paper/2023/hash/8fd1a81c882cd45f64958da6284f4a3f-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2023/hash/8fd1a81c882cd45f64958da6284f4a3f-Abstract-Conference.html).
- [28] Yu Du, Fangyun Wei, and Hongyang Zhang. AnyTool: Self-reflective, hierarchical agents for large-scale API calls. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11812–11829. PMLR, 2024. URL <https://proceedings.mlr.press/v235/du24h.html>.
- [29] Yanfei Chen, Jinsung Yoon, Devendra Singh Sachan, Qingze Wang, Vincent Cohen-Addad, Mohammadhossein Bateni, Chen-Yu Lee, and Tomas Pfister. Re-invoke: Tool invocation rewriting for zero-shot tool retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 4705–4726, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.270. URL <https://aclanthology.org/2024.findings-emnlp.270/>.
- [30] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [31] Yingli Zhou, Yaodong Su, Youran Sun, Shu Wang, Taotao Wang, Runyuan He, Yongwei Zhang, Sicong Liang, Xilin Liu, Yuchi Ma, et al. In-depth analysis of graph-based rag in a unified framework. *arXiv preprint arXiv:2503.04338*, 2025.
- [32] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. doi: 10.48550/arXiv.2404.16130. URL <https://arxiv.org/abs/2404.16130>.
- [33] Boyu Chen, Zirui Guo, Zidan Yang, Yuluo Chen, Junze Chen, Zhenghao Liu, Chuan Shi, and Cheng Yang. Pathrag: Pruning graph-based retrieval augmented generation with relational paths. In *Proceedings of the AAAI conference on artificial intelligence*, volume 40, pages 30183–30191, 2026.
- [34] Zirui Guo, Lianghao Xia, Yanhua Yu, Tian Ao, and Chao Huang. Lightrag: Simple and fast retrieval-augmented generation. *arXiv preprint arXiv:2410.05779*, 2(3), 2024.
- [35] Haoyu Huang, Yongfeng Huang, Junjie Yang, Zhenyu Pan, Yongqiang Chen, Kaili Ma, Hongzhi Chen, and James Cheng. Retrieval-augmented generation with hierarchical knowledge. *arXiv preprint arXiv:2503.10150*, 2025.
- [36] Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 7969–7992, 2023.
- [37] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*, 2023.
- [38] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 10014–10037, 2023.

- [39] Tiannuo Yang, Zebin Yao, Bowen Jin, Lixiao Cui, Yusen Li, Gang Wang, and Xiaoguang Liu. Demystifying and enhancing the efficiency of large language model based search agents. *arXiv preprint arXiv:2505.12065*, 2025.
- [40] Fangyuan Xu, Weijia Shi, and Eunsol Choi. Recomp: Improving retrieval-augmented lms with context compression and selective augmentation. In *The Twelfth International Conference on Learning Representations*, 2023.
- [41] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677, 2024.
- [42] Jinyuan Fang, Zaiqiao Meng, and Craig Macdonald. Trace the evidence: Constructing knowledge-grounded reasoning chains for retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 8472–8494, 2024.
- [43] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. Chameleon: Plug-and-play compositional reasoning with large language models. In *Advances in Neural Information Processing Systems*, 2023.
- [44] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general chinese embeddings. *arXiv preprint arXiv:2309.07597*, 2023. URL <https://arxiv.org/abs/2309.07597>.
- [45] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1410. URL <https://aclanthology.org/D19-1410/>.
- [46] OpenAI. Codex CLI. <https://developers.openai.com/codex/cli>, 2026. Accessed 2026-05-06.
- [47] OpenAI. GPT-5.2 Model. <https://developers.openai.com/api/docs/models/gpt-5.2>, 2025. Accessed 2026-05-06.
- [48] Google. Gemini CLI Documentation. <https://google-gemini.github.io/gemini-cli/docs/>, 2026. Accessed 2026-05-06.
- [49] Google Developers Blog. Gemini 3 Flash is now available in Gemini CLI. <https://developers.googleblog.com/gemini-3-flash-is-now-available-in-gemini-cli/>, 2025. Accessed 2026-05-06.

## A Appendix

### A.1 Implementation Details

The subunit extractor is deterministic. It collects three types of support evidence from each source SKILL.md: procedural lines, element-like strings such as file names, command-line patterns, and library names, and short constraint-like lines matched by requirement keywords. It normalizes text, removes duplicate subunits, filters subunits by token length, and creates skill-subunit extraction edges. Skill descriptions used for skill-level representations are extracted from the YAML frontmatter of each source SKILL.md.

For L2 community construction, KMeans uses `random_state=42` and `n_init=10`, with the number of clusters set to  $k = \lfloor \sqrt{|S|} \rfloor$ . Rescue selection uses a parent-score threshold of 0.35, a subunit score threshold of 0.12, a global rescue cap of 3, a per-parent cap of 1, and a token-Jaccard redundancy threshold of 0.6. The rescue candidate pool is bounded by

$$\min(|\text{ranked}|, \max(10, 4k)).$$

The context budget is 384 tokens.

For retrieval baselines, we use  $\text{top-}k = 5$  selected skills. The LLM-based retrieval baseline first forms a 32-skill candidate pool with embedding retrieval before LLM selection.

### A.2 Implementation Details of Affiliate Attachment

This appendix gives the implementation details behind the affiliate attachment score used in Section 3.3.2. Affiliate attachment is a deterministic post-retrieval context attachment layer: it does not change the selected skill set, does not perform random walk or graph diffusion, and does not introduce new executable skills. Given a rescued subunit  $u$  and a candidate selected skill  $s$ , the implementation scores the attachment by a fixed weighted combination of five features:

$$\begin{aligned} \text{AffScore}_{\text{impl}}(u, s \mid q) = & 0.15 q_{\text{rel}}(u, q) + 0.45 f_{\text{skill}}(u, s, H(s, q)) \\ & + 0.10 r_{\text{sel}}(s, q) + 0.15 g_{\text{same}}(u, s) + 0.15 g_{\text{active}}(u, s, K(q)). \end{aligned}$$

Here  $q_{\text{rel}}$  is token-overlap relevance between the rescued subunit text and the task request.  $f_{\text{skill}}$  is the maximum of the overlap between the rescued subunit and the selected skill profile, and the best overlap between the rescued subunit and the selected skill’s highlighted subunits.  $r_{\text{sel}}$  is the min-max normalized retrieval confidence of the selected skill.  $g_{\text{same}}$  indicates whether the rescued subunit’s source skill and the candidate selected skill belong to the same L2 capability group.  $g_{\text{active}}$  indicates whether the rescued subunit is consistent with the active L2 capability groups induced by the selected skills.

For each rescued subunit, we score all selected skills and attach the subunit to the selected skill with the highest score. The implementation then applies a small exclusivity bonus when the best selected skill is clearly separated from the second-best selected skill, and applies a penalty when the rescued subunit falls outside the active L2 capability groups. The resulting attachment is accepted only if it passes the affiliation threshold and later compact-packet gating. Compact gating controls whether an accepted affiliated cue is rendered in the final packet; it requires the cue to be concrete, task-aligned, non-redundant, and able to fit within the optional-cue token budget.

### A.3 Reproducibility Details

We use the benchmark-provided task definitions and fixed task lists for all reported experiments. SkillsBench results are reported over 87 evaluated tasks, and AgentSkillOS results are reported over 30 evaluated tasks under the benchmark score-based evaluation protocol. We do not remove tasks after observing model outputs. SkillsBench uses its deterministic verifier reward as the task outcome, and we report the macro-average reward over the fixed task set. AgentSkillOS results use the aggregate score produced by its native benchmark evaluation protocol.

**Data and skill pools.** For benchmark-native curated skills, we use the curated reference condition defined by each benchmark: per-task oracle skill folders in SkillsBench and the benchmark-defined native skill group in AgentSkillOS. For retrieval-based methods, the skill pool is loaded from the

benchmark skill repositories, where each skill is represented by a folder containing a `SKILL.md` file and optional auxiliary resources. The same task environment, skill-pool exposure protocol, runtime budget, downstream executor, and aggregation rule are used for comparable methods within each benchmark.

**Graph construction.** The multi-level skill graph is constructed offline from source `SKILL.md` files. The graph construction pipeline extracts normalized procedural, element-like, and constraint-like subunits, deduplicates them globally, and connects each retained subunit back to its source skills. Subunits and canonical skill descriptions are embedded with BAAI/bge-small-en using normalized SentenceTransformer embeddings. L2 skill communities are obtained by embedding skill representations and applying KMeans with  $k = \lfloor \sqrt{|S|} \rfloor$ , `random_state=42`, and `n_init=10`. The resulting artifacts include skill nodes, subunit nodes, skill-subunit edges, canonical skill representations, skill-community assignments, subunit identifiers, and subunit embeddings.

**Retrieval and compilation settings.** Unless otherwise stated, retrieval uses top- $k = 5$  selected skills. The retriever considers the top 30 task-matched subunits, keeps the top 2 L2 communities as the community-level signal, and exports at most three highlighted subunits per selected skill. Rescue-aware compilation uses a parent-score threshold of 0.35, a subunit score threshold of 0.12, a global rescue cap of 3, a per-parent cap of 1, and token-Jaccard redundancy filtering. Affiliate attachment uses fixed compatibility weights and accepts cues only after affiliation and compact-packet gating. The context budget is 384 tokens.

**Baselines and compilation protocol.** All retrieval-backend methods are implemented through `experiments/retrieval_tasks_backend/run_retrieval_tasks_backend.sh` and its associated mirror-preparation and aggregation scripts. Vanilla top- $k$  retrieval embeds each skill using its frontmatter name and description, ranks skills by similarity to the task request, and exposes the selected skills to the unchanged executor. LLM-based retrieval first builds an embedding-retrieval candidate pool and then asks an LLM selector to choose the final skill IDs from candidate names and descriptions. Flat retrieval baselines are additionally evaluated with a lite context protocol, which renders selected skills, task/output-contract information, and a short execution checklist into an instruction overlay without using graph-derived subunit export, rescue, or affiliate attachment. Full SkillRAE uses the graph-native context protocol implemented in the same retrieval backend: it renders selected skills, selected-skill subunit highlights, output-contract information, and affiliated rescue cues into the per-task `READ_FIRST.md / COORDINATOR_PACKET.json` payload delivered to the same executor. SkillRouter and AgentSkillOS are kept in their native forms rather than being augmented with our compiler.

**Agent/model substrates and execution.** The main comparisons use Codex CLI with GPT-5.2 [46, 47] as the downstream agent/model substrate. Component ablations additionally instantiate the same SkillRAE variants with Gemini CLI and Gemini 3 Flash. If decoding parameters are not explicitly configured by the CLI backend, we use provider defaults. The retrieval backend stages task mirrors, materializes the selected skills and context overlays, launches the benchmark executor, collects verifier outputs, and writes row-level summaries. Command-level timeouts and failure buckets are recorded in the run summaries; verifier failures after task execution are treated as benchmark failures rather than removed from the task set.

**Logging and verification artifacts.** Each run writes per-task summaries, selected-skill metadata, context artifacts, executor logs, verifier outputs, and run-level aggregation files under the corresponding output directory. These artifacts are sufficient to verify the reported SkillsBench reward means by recomputing the mean verifier reward over the fixed task list. For AgentSkillOS, we report the aggregate score from the benchmark’s native evaluation protocol.

## A.4 Compute Resources

All experiments were orchestrated on a Linux server running Ubuntu 24.04 LTS with Linux kernel 6.8.0. The worker machine has two Intel Xeon Gold 6330 CPUs with 56 physical cores and 112 logical threads in total, 1.0 TiB of RAM, Docker 29.3.0, Docker Compose v2.29.7, and Python 3.12.9. The machine also has eight NVIDIA RTX A5000 GPUs, but the inspected retrieval and graph construction code paths use CPU execution for local embedding and retrieval operations.

Table 5: Compute resources for the reported experiments. Wall-clock time is measured from available run logs and artifact timestamps when recoverable.

| Experiment                   | Wall-clock                          | Notes                                                                                                                                       |
|------------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Offline graph construction   | Not separately isolated             | Graph construction and embedding preparation are lightweight relative to agent execution and were not logged as a standalone timed phase.   |
| SkillsBench main runs        | 3.4–5.0 hours per 87-task large run | End-to-end time includes retrieval, context construction, containerized execution, verifier collection, and aggregation.                    |
| Component ablations          | 0.3–2.8 hours per 8–10 task batch   | Time varies with task subset, backend, and execution failures.                                                                              |
| Context comparison           | Not separately isolated             | Timing was not cleanly separable from the corresponding retrieval-backbone runs in the available artifacts.                                 |
| AgentSkillOS runs            | Not separately isolated             | A standalone wall-clock trace was not recovered in the bounded audit.                                                                       |
| Gemini cross-substrate check | Not separately isolated             | Gemini execution is supported by the runner configuration, but dedicated timing artifacts were not clearly separated from the sampled logs. |

The local worker handles graph preparation, retrieval, context compilation, task-mirror construction, containerized task execution orchestration, verifier execution, logging, and result aggregation. Hosted LLM inference is performed through the external Codex CLI [46] and Gemini CLI [48] backends. Runs are launched by the benchmark runners with configurable parallelism; the audited retrieval backend uses one worker by default, while some archived launch wrappers use up to four workers.

Across the bounded set of sampled run directories, we observed at least 303 planned task-level Harbor invocations and at least 33.66 wall-clock hours of recorded runs. These totals are conservative because preliminary, failed, and debugging runs were not exhaustively audited. The reported compute should therefore be interpreted as the recoverable compute for the table-supporting runs rather than the full exploratory cost of the project.